

MINI PROJECTS

1. TELCO DATA CHECK & QUERY

Write a program to perform some queries on a telco data (comming from stdin) with the following format:

The first block of data consists of lines (terminated by a line containing #), each line (number of lines can be up to 100000) is under the form:

call <from_number> <to_number> <date> <from_time> <end_time>

which is a call from the phone number <from_number> to a phone number <to_number> on <date>, and starting at time-point <from_time>, terminating at time-point <end_time>

- <from_number> and <to_number> are string of 10 characters (a phone number is correct if it contains only digits 0,1,...,9, otherwise, the phone number is incorrect)
- <date> is under the form YYYY-MM-DD (for example 2022-10-21)
- <from_time> and <to_time> are under the form hh:mm:ss (for example, 10:07:23)

The second block consists of queries (terminated by a line containing #), each query in a line (number of lines can be up to 100000) and belongs to one of the following types:

- ?check_phone_number: print to stdout (in a new line) value 1 if no phone number is incorrect
- ?number_calls_from <phone_number>: print to stdout (in a new line) the number of times a call is made from <phone_number>
- ?number_total_calls: print to stdout (in a new line) the total number of calls of the data
- ?count_time_calls_from <phone_number>: print to stdout (in a new line) the total time duration (in seconds) the calls are made from <phone_number>

Example

Input

```
call 0912345678 0132465789 2022-07-12 10:30:23 10:32:00
call 0912345678 0945324545 2022-07-13 11:30:10 11:35:11
call 0132465789 0945324545 2022-07-13 11:30:23 11:32:23
call 0945324545 0912345678 2022-07-13 07:30:23 07:48:30
```

```
#
?check_phone_number
?number_calls_from 0912345678
?number_total_calls
?count_time_calls_from 0912345678
?count_time_calls_from 0132465789
#
```

Output

```
1
2
4
398
120
```

2. CITIZEN DATA ANALYSIS

Given a DataBase about citizen, perform queries over this DataBase.

Input

The input consists of two blocks: the first block is the DataBase and the second block is the list of queries. Two blocks are separated by a line containing a character *.

1. The first block (DataBase about citizen) consists of lines, each line is the information about a person and is under the format:

<code> <date_of_birth> <father_code> <mother_code> <is_alive> <region_code>

in which:

- <code>: the code of the person which is a string of length 7
- <date_of_birth>: the date of birth of the person and has the format YYYY-MM-DD (for example 1980-02-23)

- <father_code> and <mother_code> is the code of father and mother: they are also strings of length 7. If the code is 0000000, then the current person does not have information about his father or mother
- <is_alive>: a character with two values: 'Y' means that the person is still alive, and 'N' means that the current person is died.
- <region_code>: the code of the region where the person lives

2. The second block is the list of queries over the DataBase which consists of following commands:

- NUMBER_PEOPLE: return the number of people (number of lines of the DataBase)
- NUMBER_PEOPLE_BORN_AT <date>: return the number of people having date-of-birth is equal to <date>
- MOST_ALIVE_ANCESTOR <code>: find the most ancestor (farthest in term of generation distance) of the given person <code>. Return the generation distance between the ancestor found and the given person
- NUMBER_PEOPLE_BORN_BETWEEN <from_date> <to_date>: compute the number of people having date-of-birth between <from_date> and <to_date> (<from_date> and <to_date> are under the form YYYY-MM-DD)
- MAX_UNRELATED_PEOPLE: find a subset of people in which two any people of the subset do not have father/mother-children and the size of the subset is maximal. Return the size of the subset found.

The second block is terminated by a line containing ***.

Output

- Each line presents the result of the corresponding query (described above).

Example

Input

```
0000001 1920-08-10 0000000 0000000 Y 00002
0000002 1920-11-03 0000000 0000000 Y 00003
0000003 1948-02-13 0000001 0000002 Y 00005
0000004 1946-01-16 0000001 0000002 Y 00005
0000005 1920-11-27 0000000 0000000 Y 00005
0000006 1920-02-29 0000000 0000000 Y 00004
0000007 1948-07-18 0000005 0000006 Y 00005
0000008 1948-07-18 0000005 0000006 Y 00002
```

```
0000009 1920-03-09 0000000 0000000 Y 00005
0000010 1920-10-16 0000000 0000000 Y 00005
*
NUMBER_PEOPLE
NUMBER_PEOPLE_BORN_AT 1919-12-10
NUMBER_PEOPLE_BORN_AT 1948-07-18
MAX_UNRELATED_PEOPLE
MOST_ALIVE_ANCESTOR 0000008
MOST_ALIVE_ANCESTOR 0000001
NUMBER_PEOPLE_BORN_BETWEEN 1900-12-19 1928-11-16
NUMBER_PEOPLE_BORN_BETWEEN 1944-08-13 1977-12-15
NUMBER_PEOPLE_BORN_BETWEEN 1987-01-24 1988-06-03
***
```

Output

```
10
0
2
6
1
0
6
4
0
```

3. BANK TRANSACTION

The data about bank transactions consists of a sequence of transactions: the information of each transaction has the following format:

<from_account> <to_account> <money> <time_point> <atm>

In which:

- <from_account>: the account from which money is transferred (which is a string of length from 6 to 20)
- <to_account>: the account which receives money in the transaction (which is a string of length from 6 to 20)
- <money>: amount of money transferred in the transaction (which is an integer from 1 to 10000)
- <time_point>: the time point at which the transaction is performed, it is a string under the format HH:MM:SS (hour: minute: second)
- <atm>: the code of the ATM where the transaction is taken (a string of length from 3 to 10)

Example: T00112233445 T001234002 2000 08:36:25 BIDV (at the ATM BIDV, account T00112233445 transfers 2000\$ to account T001234002 at time point 08:36:25 (08 hour, 36 minutes, 25 seconds)

A *transaction cycle* of length k starting from account a_1 is defined to be a sequence of account $a_1, a_2, \dots, a_k$ in which there are transactions from account a_1 to a_2 , from a_2 to a_3 , ..., from a_k to a_1 .

Write a program that process the following queries:

- ?number_transactions: compute the total number of transactions of the data
- ?total_money_transaction: compute the total amount of money of transactions
- ?list_sorted_accounts: compute the sequence of bank accounts (including sending and receiving accounts) appearing in the transaction (sorted in an increasing (alphabetical) order)
- ?total_money_transaction_from <account>: compute the total amount of money transferred from the account <account>
- ?inspect_cycle <account> k : return 1 if there is a *transaction cycle* of length k , starting from <account>, and return 0, otherwise

Input (stdin)

The input consists of 2 blocks of information: the data block and the query block

- The data block consists of lines:
 - o Each line contains the information about a transaction described above
 - o The data is terminated by a line containing #
- The query block consists of lines:
 - o Each line is a query described above
 - o The query block is terminated by a line containing #

Output (stdout)

- Print to stdout (in each line) the result of each query described above

Example

Input

```
T000010010 T000010020 1000 10:20:30 ATM1
T000010010 T000010030 2000 10:02:30 ATM2
T000010010 T000010040 1500 09:23:30 ATM1
T000010020 T000010030 3000 08:20:31 ATM1
T000010030 T000010010 4000 12:40:00 ATM2
T000010040 T000010010 2000 10:30:00 ATM1
T000010020 T000010040 3000 08:20:31 ATM1
T000010040 T000010030 2000 11:30:00 ATM1
T000010040 T000010030 1000 18:30:00 ATM1
#
```

```
?number_transactions
```

```
?total_money_transaction
```

```
?list_sorted_accounts
```

```
?total_money_transaction_from T000010010
```

```
?inspect_cycle T000010010 3
```

```
#
```

Output

```
9
```

```
19500
```

```
T000010010 T000010020 T000010030 T000010040
```

```
4500
```

```
1
```

4. ANALYZE DELIVERY TRIPS

The objective is to read data about delivery trips and perform some queries over this data.

The input consists of 3 blocks:

1. The first block is a sequence of consecutive dates, each date is in a line under the format YYYY-MM-DD (example 2022-01-23). The first block is terminated by a line containing *

2. The second block is about the trips. Each line describes which trip delivers some amount of packages to some customer at a date-time, and has the following format:

<trip_code> <customer_code> <date> <time> qty

in which:

- <trip_code>: code of the trip: string of length from 5-20
- <customer_code>: code of the customer: string of length from 1 to 20
- <date> <time>: the date-time on which the trip <trip_code> delivery packages to customer <customer_code>, <date> has the format YYYY-MM-DD (example 2021-10-13) and <time> has the format hh:mm:ss (example 13:37:21)
- qty: amount of packages that the trip <trip_code> delivers to customer <customer_code>

The working time period of a trip is specified from the earliest date-time to the latest date-time the trip delivers packages to customers.

The second block is terminated by a line containing ***

3. The third block is about queries. The queries consist of lines, each line is the information about one query of types:

- TOTAL_QTY: return the total quantity of packages that all the trips deliver to customers in the DataBase
- QTY_CUSTOMER <customer_code>: return the total quantity of packages that the customer <customer_code> is delivered
- QTY_MAX_PERIOD <from_date> <from_time> <to_date> <to_time>: find the date-time at which the total quantity of packages delivered is maximal. Return this maximum total quantity
- TOTAL_TRIPS: return the total number of trips
- TRAVEL_TIME_TRIP <trip_code>: return the total travel time (in seconds) of the trip <trip_code>
- MAX_CONFLICT_TRIP: compute the subset of trips in which two any trips of the subset have overlap working time period and the cardinality of the subset is maximal. Return the cardinality of the subset found.

Example

Input

2020-01-02

2020-01-03

2020-01-04

2020-01-05

2020-01-06

*

TR00000003 C000000002 2020-01-04 00:15:07 40

TR00000002 C000000001 2020-01-06 19:30:51 90

TR00000001 C000000001 2020-01-04 08:15:43 20

TR00000001 C000000003 2020-01-05 12:34:03 50

TR00000005 C000000003 2020-01-04 23:00:30 20

TR00000005 C000000001 2020-01-04 10:08:50 40

TR00000005 C000000001 2020-01-05 02:55:30 60

TR00000004 C000000003 2020-01-05 19:33:48 40

TR00000002 C000000001 2020-01-06 02:44:11 60

TR00000004 C000000001 2020-01-05 15:03:48 80

TR00000003 C000000004 2020-01-04 23:38:27 100

TR00000004 C000000001 2020-01-05 22:17:08 60

TR00000004 C000000002 2020-01-05 01:40:28 30

TR00000005 C000000002 2020-01-04 13:32:10 90

TR00000003 C000000002 2020-01-03 17:35:07 20

TR00000001 C000000001 2020-01-02 03:05:43 10

TR00000002 C000000003 2020-01-06 00:00:51 60

TR00000002 C000000002 2020-01-06 06:07:31 20

TOTAL_QTY

QTY_CUSTOMER C000000002

QTY_CUSTOMER C000000003


```
QTY_MAX_PERIOD 2020-01-02 01:04:17 2020-01-04 05:27:57
QTY_MAX_PERIOD 2020-01-02 20:21:46 2020-01-03 06:14:18
QTY_MAX_PERIOD 2020-01-02 02:58:24 2020-01-03 05:27:40
TOTAL_TRIPS
TRAVEL_TIME_TRIP TR00000003
MAX_CONFLICT_TRIPS
***
```

Output

```
890
200
170
40
0
10
5
108200
3
```

5. ANALYZE CODE SUBMISSION OF A PROGRAMMING CONTEST

Data about submission of a programming contest consists a sequence of lines, each line has the following information:

`<UserID> <ProblemID> <TimePoint> <Status> <Point>`

in which the user `<UserID>` submit his/her code to solve the problem `<ProblemID>` at time-point `<TimePoint>`, and get status `<Status>` and point `<Point>`

- `<UserID>`: string of length from 3 to 10
- `<ProblemID>`: string under the format Pxy where x, y are digits 0,1,...,9 (for example P03, P10)
- `<TimePoint>`: string representing time-point with the format HH:MM:SS (for example, 09:45:20 means the time-point 9 hour 45 minutes 20 seconds)

- <Status>: string with two cases (ERR, OK)
- <Point>: integer from {0, 1, 2, ..., 10}

A user can submit the code for solving each problem several time. The point that the user gets for a problem is the maximal point among the submissions for that problem.

Perform a sequence of queries of following types:

- ?total_number_submissions: return the number of submissions of the contest
- ?number_error_submission: return the number of submissions having status ERR
- ?number_error_submission_of_user <UserID>: return the number of submission having status ERR of user <UserID>
- ?total_point_of_user <UserID>: return the total point of user <UserID>
- ?number_submission_period <from_time_point> <to_time_point>: return the number of submissions in the period from <from_time_point> to <to_time_point> (inclusive)

Input

The input consists of two blocks of data:

The first block is the operational data, which is a sequence of lines, each line contains the information of a submission with above format .The first block is terminated with a line containing the character #

The second block is the query block, which is a sequence of lines, each line is a query described above. The second block is terminated with a line containing the character #

Output

Write in each line, the result of the corresponding query

Example

Input

```
U001 P01 10:30:20 ERR 0
U001 P01 10:35:20 OK 10
U001 P02 10:40:20 ERR 0
U001 P02 10:55:20 OK 7
U002 P01 10:40:20 ERR 0
```

U001 P01 11:35:20 OK 8

U002 P02 10:40:20 OK 10

#

?total_number_submissions

?number_error_submission

?number_error_submission_of_user U002

?total_point_of_user U001

?number_submission_period 10:00:00 11:30:45

#

Output

7

3

1

17

6

6. ANALYZE SALES ORDERS OF AN E-COMMERCE COMPANY

Data about sales in an e-commerce company (the e-commerce company has several shops) consists a sequence of lines, each line (represents an order) has the following information:

<CustomerID> <ProductID> <Price> <ShopID> <TimePoint>

- in which the customer <CustomerID> buys a product <ProductID> with price <Price> at the shop <ShopID> at the time-point <TimePoint>
- <CustomerID>: string of length from 3 to 10
- <ProductID>: string of length from 3 to 10
- <Price>: a positive integer from 1 to 1000
- <ShopID>: string of length from 3 to 10
- <TimePoint>: string representing time-point with the format HH:MM:SS (for example, 09:45:20 means the time-point 9 hour 45 minutes 20 seconds)

Perform a sequence of queries of following types:

- ?total_number_orders: return the total number of orders
- ?total_revenue: return the total revenue the e-commerce company gets
- ?revenue_of_shop <ShopID>: return the total revenue the shop <ShopID> gets
- ?total_consume_of_customer_shop <CustomerID> <ShopID>: return the total revenue the shop <ShopID> sells products to customer <CustomerID>
- ?total_revenue_in_period <from_time> <to_time>: return the total revenue the e-commerce gets of the period from <from_time> to <to_time> (inclusive)

Input

The input consists of two blocks of data:

The first block is the operational data, which is a sequence of lines, each line contains the information of a submission with above format

The first block is terminated with a line containing the character #

The second block is the query block, which is a sequence of lines, each line is a query described above

The second block is terminated with a line containing the character #

Output

Write in each line, the result of the corresponding query

Example

Input

```
C001 P001 10 SHOP001 10:30:10
C001 P002 30 SHOP001 12:30:10
C003 P001 40 SHOP002 10:15:20
C001 P001 80 SHOP002 08:40:10
C002 P001 130 SHOP001 10:30:10
C002 P001 160 SHOP003 11:30:20
```

#

?total_number_orders

?total_revenue

?revenue_of_shop SHOP001

?total_consume_of_customer_shop C001 SHOP001

?total_revenue_in_period 10:00:00 18:40:45

#

Output

6

450

170

40

370